4. News Feed (Twitter / Instagram)

> **Interviewer:** Let's design a news feed — think Twitter or Instagram. Users post content, follow other users, and open an app to see a home timeline of what the people they follow have posted, roughly newest-first. Design the backend.

Candidate: Great, this is a fan-out problem at heart. Let me clarify scope before I size anything, because the shape of the social graph is going to drive the whole design.

---

1. Clarifying the requirements

> **Candidate:** A few questions: 1. Reverse-chronological, or ranked ("best" posts first, like Instagram's algorithmic feed)? 2. Do follower counts follow a normal social distribution, or do we need to handle **celebrities with tens of millions of followers**? 3. Can a follower tolerate seeing a new post a couple of seconds late, or must it be instant? 4. Media (images/video) — stored and served how, or out of scope?
>
> **Interviewer:** Start with reverse-chronological, mention ranking as an extension. Yes — assume the full range, including a handful of accounts with **100 million followers**. A couple of seconds of delay is completely fine. Media exists but treat it as "stored somewhere, referenced by URL" — don't design the video pipeline.

Candidate: Good, that focuses things. Requirements:

Functional - `post(userId, content)` → create a post. - `follow(userId, targetId)` / `unfollow(...)`. - `getFeed(userId, cursor)` → paginated home timeline, newest-first, of everyone the user follows.

Non-functional - **Read-heavy**: feed opens vastly outnumber posts — this is a read-scaling problem wearing a write-scaling costume. - **Feed reads must be fast**, sub-200ms — it's the app's home screen, hit constantly. - **Writes must never be lost**, but they can be *delayed* — eventual consistency is fine. - **The celebrity problem is the whole ballgame**: a user with 100M followers posting once must not mean 100M synchronous writes on the critical path.

---

2. Estimation

```
200M daily active users, avg 300 followers/user, but a long tail up to 100M followers.

Posts/day        = 50M          -> ~600 writes/sec avg (peak ~3k)
Feed opens/day    = 200M * 10   = 2B  -> ~23,000 reads/sec avg (peak ~100k)
Read : write ratio ~ 40:1        -- reads dominate; optimize reads above all else.

The number that defines this problem — fan-out cost if EVERY post is pushed to
EVERY follower's feed at write time:

  normal post:     600/sec * 300 followers        =  180,000 feed-inserts/sec  (fine)
  ONE celebrity post, 100M followers:
      100,000,000 feed-inserts for a SINGLE post
      at even 100k inserts/sec of fan-out capacity -> 1,000 seconds (~17 min)
      just to finish delivering ONE tweet.

=> Pure "push everything" cannot work for celebrities. Pure "compute everything
   at read time" cannot work for normal users at 40:1 read:write. We need both,
   applied selectively. That selection is the core design decision. (Section 4.)

Storage: 50M posts/day * 365 * 5yr ~= 90B posts, ~1KB metadata each -> ~90 TB over
   5 years (media stored separately in blob + CDN).
Feed cache: 200M users * 800 cached post-IDs * 8 bytes ~= 1.3 TB in Redis — fits a
   cluster comfortably.
```

3. V1 — the naive design (single region, ignore celebrities for now)

> **Candidate:** Let me put the simplest possible thing on the table first, then show exactly where it breaks.

```
  [ Client ] --POST /post--> [ Post Service ] --write--> [ Posts table ]
                                                          (author_id, content, ts)

  [ Client ] --GET /feed---> [ Feed Service ] --> for each followee of user:
                                                     SELECT posts WHERE author_id = ?
                                                     ORDER BY ts DESC LIMIT 20
                                                  --> merge-sort in app code --> return
```

Why it dies at scale: every feed open does N queries (one per followee) plus an in-app merge. A user following 500 accounts triggers 500 lookups *per refresh*, at 100k reads/sec peak — tens of millions of author-lookups per second. That's fan-out-on-read, and it's catastrophic for a 40:1 read-heavy workload. We need to precompute.

4. The centerpiece — fan-out on write vs. fan-out on read

> **Interviewer:** Right, so precompute. Walk me through how.

Candidate: This is the fundamental fork in every feed system: **do the assembly work at post time (push), or at read time (pull)?**

Fan-out on write (push)

```
User A posts (300 followers)
    |
    ▼
[ Fan-out worker ] ── for each follower F of A ──► prepend postId to
    |                                                feed:{F} in Redis
    ▼
feed:follower_1 = [ newPostId, older, older, ... ]
feed:follower_2 = [ newPostId, older, older, ... ]
...
feed:follower_300 = [ newPostId, older, older, ... ]

READ: GET feed:{userId}  -> already-built Redis list -> O(1). FAST.
```

- **Pro:** reads are a single key lookup. This is exactly what a 40:1 read-heavy workload wants — push the cost to the rare event (a post), not the frequent one (a read).
- **Con: write amplification.** One post → one write per follower. Fine at 300 followers; catastrophic at 100 million.

Fan-out on read (pull)

```
READ: user U follows {A, B, C, ...}
  getFeed(U)
    -> look up U's followee list                (graph store)
    -> fetch recent posts for A, B, C, ... in parallel
    -> merge by timestamp, paginate, return

POST: write the post once. O(1). Cheap, no matter who posts.
```

- **Pro:** writes are trivial regardless of follower count — no celebrity problem on write.
- **Con:** reads become the expensive operation, and reads are 40x more frequent than writes. Wrong tradeoff for the common case.

The hybrid — what real systems (Twitter's actual design) do

> **Interviewer:** So which one do you pick?

Candidate: Neither, alone — I pick **both, applied to different populations of authors**, because the two failure modes are mirror images of each other and the split of "normal user" vs. "celebrity" happens to line up perfectly with which model each needs.

```
Regular user posts   ──►  FAN-OUT ON WRITE   (push postId into every follower's
                                               Redis feed list — cheap, follower
                                               count is small)

Celebrity posts      ──►  DO NOT FAN OUT.     Just durably store the post.
(>1M followers, say)      (Pushing to 100M feeds would take ~17 minutes and
                           hammer the fan-out pipeline for one tweet.)

getFeed(U):
  1. Read U's precomputed feed:{U} from Redis  -- covers every NORMAL followee.
  2. Read U's (small) list of followed celebrities -- pull THEIR recent posts
     directly from the post store, on demand.
  3. Merge (1) + (2) by timestamp -> paginate -> return.
```

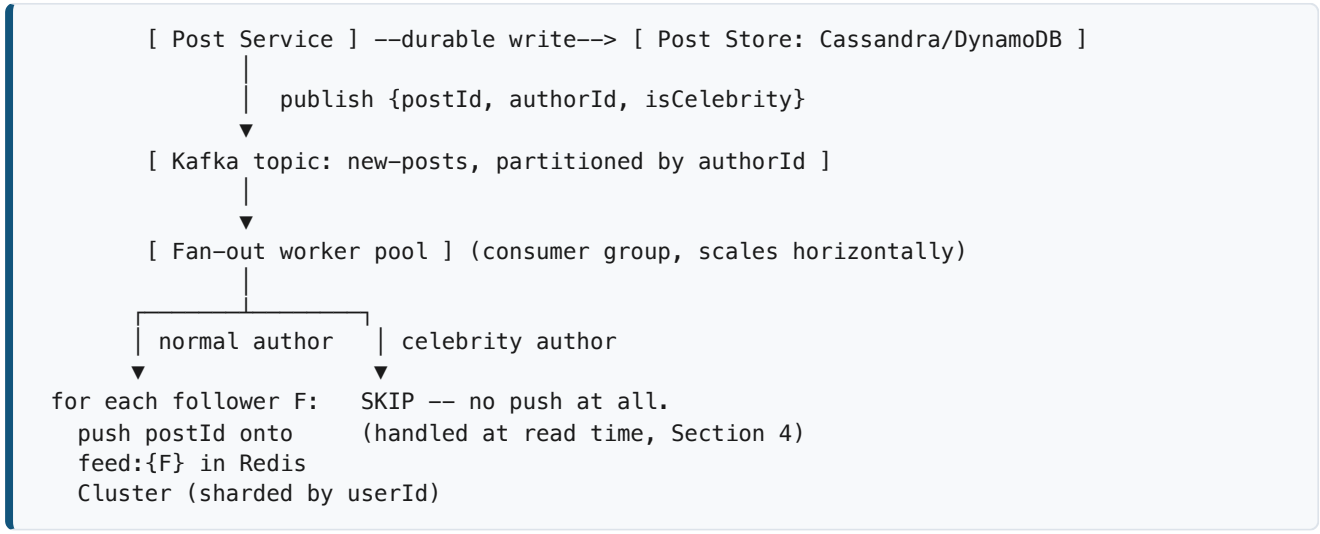
- The **threshold** ("celebrity" = follower count above, say, 1M) is enforced on the `follow` edge itself — when U follows a celebrity, we tag that edge `is_celebrity_edge = true` so fan-out workers know to skip it.
- **Why this wins:** the overwhelming majority of accounts are normal, so push keeps the common-case read at O(1). Celebrity accounts are rare, so even though *many* users follow them, each of those users only pulls from a *handful* of celebrities at read time — cheap, because it's bounded by "how many celebrities does U follow" (typically single digits to low dozens), not "how many total followees does U have."
- This is the answer interviewers are listening for — say the hybrid explicitly, don't just pick one side.

---

5. Scaling further — hundreds of millions of users

> **Interviewer:** Good. Now push this to hundreds of millions of users globally. What breaks first, and what do you change?

Candidate: Three things start to strain: **the follower graph** (partition it by `followee_id` so "give me all followers of A" is a single-partition scan, not a scatter-gather); **fan-out throughput** (needs a horizontally scalable worker pool, not one process looping over followers); and **Redis feed-cache capacity** (1.3 TB across 200M users needs a real Redis Cluster, sharded by `userId`, so no single node is overloaded).



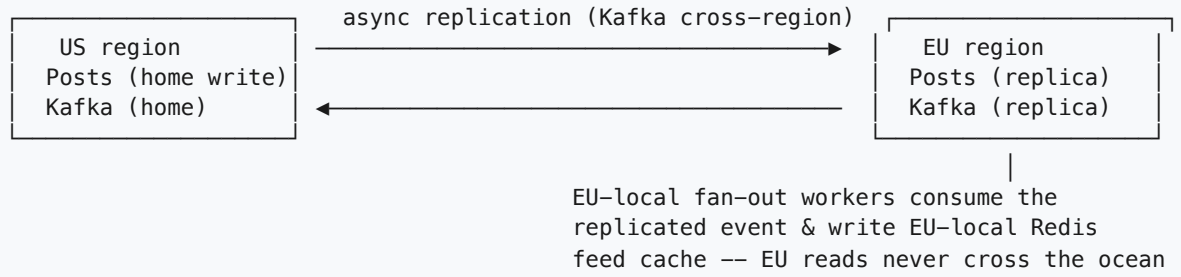
The key scaling move: **the fan-out pipeline is horizontally scalable because Kafka partitions by `authorId` and a consumer group just adds workers** — throughput grows with worker count, with no shared mutable state between workers.

---

6. Multi-region

> **Interviewer:** Now make it global — users in the US, EU, and APAC, all following each other across regions.

Candidate: I'd home each user's write path (their own posts, their own follow graph edits) in one **home region**, close to where they mostly use the app, and replicate asynchronously outward.



- A post is written once, in the author's home region, to the durable post store — the single source of truth. It replicates asynchronously to other regions, where **local** fan-out workers push it into **local** Redis caches.
- **Cross-region edge case:** a US celebrity followed by EU users — the pull-at-read-time step now either hits a replicated read-copy in EU or crosses region if replication hasn't caught up. Either way this only adds latency, never incorrectness — feeds are already eventually consistent, so a post landing a second or two later for cross-region followers is invisible in practice. We don't chase "instant everywhere" — that's a global-consensus cost the product doesn't need.

---

7. Latency — where the milliseconds go

> **Interviewer:** You said feed reads need to be under 200ms. Walk me through a single `getFeed` call and tell me where time goes.

Candidate:

```
getFeed(U)                                                    budget
  1. Auth / gateway routing                                    ~2ms
  2. Redis GET feed:{U}  (sorted set, top 20)                  ~2–5ms   <- the fast path
  3. Redis GET U's celebrity-follow list (small, cached)        ~1ms
  4. For each followed celebrity (typically < 10):
       pull their latest posts (also Redis-cached per author)   ~5–10ms, done in PARALLEL
  5. Merge in app code, paginate                               ~1–2ms
  6. Hydrate author profile summaries (name/avatar) from a
     profile cache, batched in one multi-get                    ~5ms
  ------------------------------------------------------------------------
  total, cache-hot path                                       ~20–30ms, well under budget
```

- **Cache miss on step 2** (new device, evicted key) is the slow path — we fall back to pulling recent posts from every followee directly from the post store and rebuilding the Redis feed before returning. That's the one case that can approach the 200ms ceiling; it's rare, so it's an acceptable outlier, not the steady state.
- **The recurring theme:** everything on the hot read path is an in-memory key lookup, not a disk-backed query. That's *why* fan-out-on-write exists — it pays the assembly cost once at write time instead of on every one of the 40x-more-frequent reads.

---

8. Consistency — what's strong, what's eventual

- **Strong-ish (read-your-own-writes):** a user who posts should see their own post in their own profile/feed immediately — the post write and the author's own feed insert happen synchronously enough for that to hold.
- **Eventual everywhere else:** whether *your followers* see your post in the next 100ms or the next 3 seconds is explicitly acceptable — that's the async fan-out lag, and there is no requirement for global real-time delivery.
- **Why this is the right tradeoff:** a feed is not a financial ledger. Nobody is harmed by a post arriving a couple of seconds late; users *are* harmed by a slow or unavailable app. We spend the consistency budget on availability and latency, not on synchronizing every follower's view instantly.

---

9. Async — the fan-out pipeline off the critical path

> **Interviewer:** Make sure "post" itself stays fast even for a celebrity. What's on the critical path of the `POST /post` call?

Candidate: As little as possible.

```
Client --POST /post--> Post Service
    |
    ├─ write post to durable store (Cassandra/DynamoDB)  -- this MUST succeed
    ├─ publish {postId, authorId, isCelebrity} to Kafka "new-posts"
    └─ return 200 to client  ◄── IMMEDIATELY. Fan-out has NOT happened yet.

Kafka "new-posts" (partitioned by authorId, durable, replicated)
    |
    ▼
[ Fan-out worker pool ]  -- consumes asynchronously, off the critical path
    if normal author: push postId into every follower's Redis feed list
    if celebrity author: do nothing here (handled at read time)
```

- The user who tapped "post" waits only for a durable write + a queue publish — both fast, local operations. They are **never** blocked on "how many followers does this person have," which is exactly the variable that made push dangerous in the first place.
- **Why a queue, not a direct call from the post service to a fan-out function?** Fan-out volume is bursty and unbounded at any instant (many posts can land at once). A queue absorbs bursts, lets fan-out workers scale independently of the write path, and gives durable retry: if a worker crashes mid-fan-out, Kafka redelivers instead of losing a job that only lived in a request thread's memory.
- **Idempotency matters here too.** A crash-and-retry, or an at-least-once redelivery, must not double-insert a `postId` into a follower's feed. A Redis **sorted set keyed by postId** (score = timestamp) makes re-insertion a safe no-op.

---

10. Caching — what, where, invalidation

> **Interviewer:** You've leaned hard on Redis. What exactly is cached, and what happens on a cache miss or eviction?

Candidate:

```
What lives in Redis:
  feed:{userId}     -- sorted set of postId, score = timestamp/SnowflakeID,
                       trimmed to newest N (~800) -- nobody scrolls further back
  recent:{authorId}  -- small cache of an author's latest posts, used by the
                       celebrity PULL path -- shared/reused across the millions
                       of users who follow the same celebrity
  profile:{userId}   -- name/avatar for hydration, short TTL

Cache-aside on miss:
  feed:{U} miss (new user, evicted key)
    -> pull recent posts from ALL of U's followees directly from the post store
    -> merge, cache the rebuilt list into feed:{U}, THEN return  (slow once)

Invalidation:
  feed lists are append/trim only -- no explicit invalidation in normal flow.
  A tombstone marks a removed post (filtered at read time) rather than
  scrubbing millions of follower lists synchronously.
```

- **Why Redis, not the primary post store, for this hot structure:** we need O(1) reads-by-user and cheap prepend/trim — an in-memory sorted-set job, not a system-of-record's job. Redis is the

source of truth for *feed order* only; posts themselves live durably elsewhere and the Redis structure is always rebuildable from them. Losing the cache is a performance incident, not a data-loss incident.

11. Technology choices — what and why

Concern	Choice	Why this, and why not the alternative
Post store (source of truth)	**Cassandra / DynamoDB**, partitioned by `author_id`, clustered by time-sortable ID	Write pattern is "write once, read by ID or by author, at huge scale, no joins/transactions needed." Wide-column stores scale writes horizontally by adding nodes; partition-by-author + cluster-by-time gives "latest posts by this author" as a single-partition range scan — exactly what the celebrity pull path needs. *Not a relational DB* — at 600+ writes/sec sustained (bursting far higher) with no cross-post transactions, a single-writer-friendly relational schema is harder to shard and buys us nothing ACID-wise that we actually need.
Post IDs	**Snowflake-style IDs** (`timestamp | worker | sequence`)	Time-sortable without a central counter or cross-node coordination — critical at this write volume. *Not auto-increment* — a single incrementing counter is a global bottleneck and a single point of failure.
Precomputed feed cache	**Redis, sorted set per user** (`feed:{userId}`)	Sub-millisecond, O(1) reads by key, natural "ordered list with score = timestamp" structure, cheap prepend + trim, and re-insertion of the same postId is a safe idempotent no-op. *Not the post store itself for this* — a durable wide-column store isn't built for "give me this one user's precomputed ordered list" at 100k+ reads/sec; that's a cache's job.
Social graph (follows)	**Adjacency-list tables** (`follower_id`, `followee_id`, indexed both directions), with an `is_celebrity_edge` flag	The core queries are "who does U follow" and "who follows A" — a simple bidirectional adjacency list serves both without needing multi-hop graph traversal. *Not a dedicated graph DB (Neo4j-style)* — we never need "friends of friends" or path queries here; that machinery is unjustified complexity for two lookup patterns.
Fan-out pipeline	**Kafka**, topic partitioned by `authorId`	Durable, replayable, ordered-per-partition log that decouples "a post happened" from "fan-out finished." Lets the post-write path return instantly and lets worker count scale independently to absorb bursts (including — deliberately — never processing celebrity fan-out at all). *Not a direct synchronous call from the post service* — that reintroduces "the post API's latency depends on follower count," the exact thing we're trying to eliminate. *Not a simple SQS-style queue* — we want partitioned ordering per author and the option to replay if a worker pool needs to catch up or reprocess.
Celebrity threshold	**Flag on the follow edge** (`is_celebrity_edge`),	Cheap to check per fan-out event (one boolean), computed once rather than re-evaluating follower-count

Concern	Choice	Why this, and why not the alternative
enforcement	checked once at follow-time	on every post.
Media (images/video)	**Object storage (S3) + CDN**, DB stores only a URL	Binary blobs don't belong in a database; CDN handles the read-heavy, geographically-distributed nature of media delivery far better than origin-serving from application servers.
Client-facing feed API	**GraphQL**	A single feed screen needs post content + author profile + media URL + like count in one response — a nested, multi-entity fetch. GraphQL lets different clients (mobile vs. web) shape the query without a bespoke endpoint per client. *Not plain REST* — we'd end up hand-rolling a "feed with embeds" endpoint anyway, which is GraphQL by another name minus the flexibility.
Internal service calls	**gRPC** (feed service ↔ fan-out workers ↔ graph service ↔ post service)	High-volume, service-to-service, benefits from typed contracts and low overhead; REST/GraphQL's client-facing flexibility isn't needed internally.
Load balancing	**L7 / API Gateway**, routed by path	Post creation, follow mutations, and feed reads have wildly different load profiles (reads dominate 40:1) and need independently-scaled fleets — that requires path-based routing, which is an L7 concern. Also where we enforce per-user post rate limits (anti-spam).

Say-it-out-loud justification: "*The core tension is fan-out on write vs. fan-out on read — push gives O(1) reads but blows up on celebrity write-amplification; pull gives cheap writes but expensive reads for anyone who follows many accounts. I use a hybrid: push for normal users, since they're the vast majority and reads outnumber writes 40:1; pull for celebrities, since even though many people follow them, each follower only pulls from a handful of celebrity accounts at read time. Posts live in Cassandra for write-scalable durable storage with Snowflake IDs for coordination-free time-ordering; the social graph is a simple bidirectional adjacency list; precomputed feeds live in Redis sorted sets, which is the one structure built for O(1) per-user ordered reads. New posts go through Kafka so fan-out is fully asynchronous and never blocks the post action — the post API's latency must never depend on how many followers the author has.*"

---

12. Failure modes & graceful degradation

Failure	What happens	Mitigation
Fan-out workers fall behind (Kafka backlog)	Posts still durable in Cassandra + queued in Kafka; followers just see the new post a bit late	Feed delivery degrades from "fast" to "eventual," never to "lost." Scale out workers to catch up.
Redis feed cache miss / node eviction	One user's feed:{U} is gone	Rebuild on demand by pulling that user's followees' recent posts from Cassandra — slower once, then re-cached. No data loss, because Redis was never the source of truth.

Failure	What happens	Mitigation
Celebrity posts (write-side hot key)	N/A — by design	Celebrities are excluded from push fan-out entirely, so there is no write hot-spot to mitigate.
Celebrity posts (read-side hot key)	Millions of users pulling the same author's recent posts	`recent:{authorId}` is itself cached — many followers reading the *same small set* of posts is a textbook cache-friendly pattern, unlike millions of distinct writes.
Fan-out worker crash mid-batch	Some followers not yet updated	Kafka redelivers the event (at-least-once); idempotent sorted-set inserts make the retry safe — no duplicates, no lost followers.
A Cassandra node down	—	Replication factor 3 + quorum reads/writes keeps serving with no visible impact.
Cross-region replication lag	EU follower sees a US celebrity's post a few seconds late	Acceptable — feeds are already eventually consistent; not treated as an incident.

Design principle: a post must never be lost, but it's completely fine for it to arrive in a follower's feed a little late. That slack is what turns queue backlogs and cache rebuilds into non-events instead of outages.

---

13. Follow-up curveballs

> **Interviewer:** What if I want ranking, not just reverse-chronological?

Candidate: The feed becomes a **candidate generation + scoring** pipeline instead of a pure merge. Pull a larger candidate set than we'll show (from the same push/pull hybrid sourcing — Redis for normal followees, direct pull for celebrities), score each candidate with an ML model (recency, engagement prediction, affinity to the viewer), then return the top-N. The *sourcing* layer barely changes; we add a scoring stage on top of it.

> **Interviewer:** A user follows 10,000 accounts. What happens to their feed?

Candidate: That's the same skew problem in reverse — a "power follower" instead of a power *followee*. Even the push path doesn't fully save us here: their `feed:{U}` list is still just the last ~800 pushed post-IDs, so it works fine for reads, but *writes* into their feed happen 10,000x more often than an average follower's (every one of the 10,000 accounts they follow fans out to them). Some systems cap follow counts; others treat very heavy-followers as needing a partial pull-and-merge instead of relying purely on the pushed list. Worth naming as an edge case, not a blocker.

> **Interviewer:** How do you cap unbounded growth of the Redis feed lists?

Candidate: Trim each `feed:{userId}` sorted set to the most recent N (e.g., 800) postIDs on every insert — nobody scrolls back further than that in practice. Older history, if needed, falls back to a direct pull from the post store.

> **Interviewer:** How do you delete a post that's already been fanned out to millions of feeds?

Candidate: Don't try to scrub it out of every Redis list synchronously — that's the same write-amplification problem as the celebrity post itself. Instead, tombstone the `postId` in the post store; the feed-read path filters out tombstoned IDs when it hydrates content. The stale ID sits harmlessly in Redis lists until it's trimmed out naturally.

---

14. Deep-dive: why not pure fan-out-on-write, or pure fan-out-on-read, for *everyone*?

> **Interviewer:** You jumped straight to "hybrid." Play devil's advocate with yourself — why can't I just pick ONE of the two models and apply it uniformly to every user? Simpler system, one code path.

Candidate: Because the two pure models fail in exactly opposite ways, and both failures are severe enough that neither survives contact with the real follower-count distribution. Let me quantify each, then justify precisely where I draw the hybrid's line.

Pure fan-out-on-write — dies on write amplification

Every post becomes `F` feed-inserts, where `F` is the author's follower count. That's fine when `F` is in the hundreds. It is not fine at the tail:

```
normal post,    300 followers:        300 inserts     -- trivial
celebrity post, 100,000,000 followers: 100,000,000 inserts

at a shared fan-out pool doing ~100K inserts/sec (Section 2's own number):
  100,000,000 / 100,000  =  1,000 sec  (~17 minutes)

That's not just "one slow tweet" -- for those 17 minutes this ONE post is
consuming the entire shared worker pool's capacity, which means every OTHER
author's normal-tier fan-out (running through the same pool) backs up too.
One celebrity post degrades delivery for millions of unrelated posts.
```

Pure fan-out-on-read — dies on the 40:1 read:write ratio

Flip it: never precompute, always assemble at read time. Now every one of the ~100K feed reads/sec (peak) does a scatter-gather across every followee:

```
user follows 500 accounts, opens the app:
  getFeed() -> 500 parallel lookups + an in-app merge-sort, EVERY refresh

at 100K feed-opens/sec peak, average 300 followees:
  100,000 * 300 = 30,000,000 author-lookups/sec, sustained, forever

Unlike the celebrity case above, this cost can't be shared across users --
every user follows a DIFFERENT set of accounts, so there's no single cache
entry that serves more than one person. It's the worst possible shape for
a 40:1 read-heavy workload: you pay the expensive operation on the side of
the ratio that fires 40x more often.
```

Comparison table

Dimension	Fan-out-on-write (pure)	Fan-out-on-read (pure)	Hybrid (this design)
Write cost per post	O(followers) — one post → `F` inserts	O(1) — just persist the post	O(followers) for normal authors; **O(1)** for celebrities (skip push)
Read cost per feed open	O(1) — single cached key	O(followees) — full scatter-gather, every refresh	O(1) push-read + O(celebrities followed), typically < 10
Which side of the 40:1 ratio pays	The rare side (writes) — correct direction	The frequent side (reads) — wrong direction	The rare side, for the common population
100M-follower author posts	~17 min write storm, starves the shared pool for everyone (see above)	No burst — it's one row	No burst — celebrities are excluded from push entirely (Section 4)
Cacheability of the expensive step	N/A — the write *is* the cache (`feed:{U}`)	Poor — every user's followee set differs; nothing is shareable	Pull side is shareable: millions of a celebrity's followers query the *same* `recent:{authorId}` — one cache entry, many readers
Fails when	Any single author's `F` is large relative to fan-out throughput	Fan-in is large — i.e. almost always, since reads dominate 40:1	Only if the split threshold misclassifies an account (see below)

Justifying the threshold precisely, not just "say, 1M"

> **Interviewer:** Fine, hybrid wins. But where exactly does "celebrity" start? You said "1M" earlier — defend that number.

Candidate: It comes from the same 100K-inserts/sec pool used above. Backlog duration for a single post is simply `followers / 100,000`:

```
followers        backlog to clear fan-out (shared 100K/s pool)
-----------      ---------------------------------------------
300                0.003 sec   -- invisible
300,000             3 sec      -- inside the "eventual, a few seconds" budget (Sec 8)
1,000,000          10 sec      -- still comfortably inside budget, even 2-3x
                                  slower if the pool is busy with other posts
10,000,000        100 sec (~1.7 min)  -- now visibly degrading, and stealing
                                         capacity from every unrelated post
                                         queued behind it in the same pool
100,000,000     1,000 sec (~17 min)  -- outage-shaped
```

There's a knee in that curve somewhere between a few hundred thousand and a couple million followers, where a single post stops being "invisible" and starts "measurably starving the shared pool." **1M sits inside that knee, on the conservative side, and that's deliberate** — the two misclassification directions are *not* symmetric in cost:

- Misclassify a large-but-not-celebrity account as "celebrity" too early → its followers pay one extra cheap, bounded pull lookup per feed open. Barely measurable.
- Misclassify a true celebrity as "normal" too late → a multi-minute write storm that degrades the *entire* shared fan-out pool, not just that author's followers.

When one direction of error is "slightly more expensive read" and the other is "shared outage," you round the threshold toward the cheap mistake. That's why 1M, not the exact breakeven — it's a safety margin against classification lag, correlated celebrity posts (many large accounts posting during the same live event, dividing the pool's effective capacity), and follower-count growth between whenever the flag was last recomputed and now.

> **Rule of thumb:** *Pick the model per the cost that scales with the dangerous variable. Fan-out-on-write is safe exactly as long as `followers × posts` stays bounded; fan-out-on-read is safe exactly as long as the query result is shareable across readers. Celebrities blow up the first and satisfy the second — that's not a coincidence, it's the whole reason the split exists. Set the threshold below the analytic breakeven, not at it, because under- and over-classifying an account cost the system very different amounts.*

---

15. Deep-dive: the celebrity fan-out spike meets a follow/unfollow race

> **Interviewer:** Let's stress-test the boundary itself. A massive account posts — say it's right at the edge of your celebrity cutoff, or the classification job hasn't caught up yet — and its fan-out job is mid-flight when a user follows or unfollows it. Who ends up seeing what, and is that correct?

Candidate: This is the sharpest edge case in the whole design, because it combines a *throughput* problem (the queue backing up) with a *correctness* problem (the follower list changing out from under an in-flight job) — and they make each other worse.

Why the queue backs up

A push fan-out job doesn't happen instantaneously — a worker pages through the follower list and pushes to each one. For a large account, that scan itself takes real wall-clock time:

```
Account with 900K followers posts (just under the 1M cutoff -- still push-tier)

[ Kafka: new-posts, partition=authorId ]
       |  one job: "fan out postId=P to all followers of A"
       v
[ Fan-out worker ]
  cursor-scans followers table in pages of 1,000:
    page 1   [ f0001 .. f1000 ]   --> push P onto each feed:{f}
    page 2   [ f1001 .. f2000 ]   --> push P onto each feed:{f}
    ...                                    (900 pages, ~seconds each
    page 900 [ f899001..f900000] -->  under load)             |
                                                              v
                                            total scan time: tens of seconds
                                            to a couple minutes if the shared
                                            pool is also busy with other posts
```

If several such accounts post close together (a live event), their jobs interleave on the same shared worker pool and *each* job's scan stretches longer — this is the queue "backing up" the interviewer is pointing at: not one infinite backlog, but every in-flight scan getting slower, which widens the race window below.

The race: a follow-edge changes while the scan is mid-flight

```
t=0s     A posts P.  Fan-out job starts.  Follower snapshot begins at page 1.
t=5s     Worker is somewhere around page 40  (followers f039001..f040000)
t=6s     User X (already pushed, back at page 12) UNFOLLOWS A.
             -> X already has P in feed:{X}. Nothing removes it.
             -> X sees a post from someone they just unfollowed.  (stale delivery)

t=7s     User Y FOLLOWS A for the first time.
             -> Y is a BRAND NEW edge. Was Y in the cursor's snapshot?
                - If the scan re-queries "current followers" per page: Y might land
                  in a page not yet reached -> Y GETS P.  (lucky, order-dependent)
                - If the scan used a fixed snapshot taken at t=0: Y is simply not
                  in it -> Y NEVER gets P pushed, silently, permanently.
                  (this is the dangerous outcome -- not late, just gone)

t=8s     User Z's follow of A causes A's follower count to tick past the 1M
          celebrity cutoff mid-scan.
             -> going forward Z (and everyone after) is tagged is_celebrity_edge
             -> but the PUSH job that's already running doesn't know that;
                it keeps pushing to whatever the cursor sees
             -> meanwhile the PULL path (recent:{authorId}) for newly-celebrity
                A may not be warmed/consistent yet -- a brief window where a
                follower is neither pushed to (now flagged celebrity) nor
                reliably served by pull (cache not yet primed)
```

Three distinct failures, worth naming separately: **(a) stale delivery** to an unfollower — harmless, self-heals when the trimmed feed list ages out; **(b) silent permanent miss** for a brand-new follower whose edge appeared after the snapshot was taken — this is the real bug, because unlike a delay, the user simply never sees that post; **(c) a classification-boundary gap** where an account flips tiers mid-job and neither path clearly owns the delivery.

Ranked fixes

1. **Pull-merge at read time for celebrities/large accounts (best, and already the design's instinct) — extend it.** For any account above the cutoff, there is no snapshot to go stale, because every read re-checks the live `is_celebrity_edge` list and re-pulls the author's latest posts fresh. No push job, no cursor, no race. This is *why* the hybrid pushes celebrities entirely into the pull lane instead of trying to make push safe for them — the race in this section only exists for push-tier accounts near the boundary.
2. **Async fan-out with bounded workers + backpressure, made idempotent and resumable.** Page through followers with a checkpointed cursor (resume-safe if a worker crashes mid-scan); cap concurrent large-account jobs per pool so one spike can't starve the others; and if queue lag crosses a threshold, temporarily degrade that author to "pull fallback" for *new* readers rather than letting the backlog grow unbounded. This bounds the damage from fix (3)'s remaining gap, it doesn't remove the race by itself.
3. **Capture the follower-set at post time, but patch the miss with a short pull-merge safety net — don't try to make the snapshot always right.** Snapshotting at post time is actually the *correct* semantic for existing followers ("who followed me when I posted" is a reasonable definition, and it's why the stale-delivery case in (a) is fine to leave as-is). The bug is specifically new followers missing the snapshot entirely. Fix cheaply: at read time, in addition to the precomputed `feed:{U}`, also pull-check any followee `U` followed within roughly the last few minutes (a tiny, bounded list) — this catches exactly the "I followed A right as A posted" case without redesigning the push path, and costs nothing for the 99.9% of follows that aren't racing a fan-out job.

Recommendation: lean on (1) — pull-merge — for everything above the celebrity cutoff, since that removes the race structurally for the population where the spike is largest. For push-tier accounts near the boundary, combine (2)'s bounded/checkpointed workers with (3)'s "recent follow" pull-merge safety net; that turns a silent permanent miss into, at worst, a feed that's briefly assembled from two sources instead of one — still O(1)-ish, still fast, and correct.

> **What to say out loud:** "*The write-side spike and the follow/unfollow race are really the same root cause: a push fan-out job takes real time to scan a large follower list, and anything that happens during that scan — an unfollow, a new follow, even the account crossing the celebrity threshold — is a race against a snapshot that's already in flight. Unfollowing mid-scan is harmless and self-heals. A brand-new follower missing the snapshot entirely is the real bug, because it's a silent, permanent miss, not a delay. I'd close it by leaning harder on the pull side: celebrities already skip push and re-check live state on every read, which structurally can't race; for push-tier accounts near the boundary, I'd add a small pull-merge safety net for anyone followed in the last few minutes, so a mistimed follow degrades to 'read from two sources' instead of 'never see the post.'*"

Summary — the mental model

A news feed is **not one problem, it's two, pulling in opposite directions: writes want to be cheap (post once), reads want to be cheap too (40:1 more frequent than writes), and a naive "precompute everything" design blows up the instant a user has 100 million followers.** The winning move is the

hybrid fan-out: push normal users' posts into precomputed Redis feed lists at write time (keeps the common-case read at O(1)), and skip push entirely for celebrities, pulling their posts at read time instead — cheap because each reader only pulls from a handful of celebrities, not thousands of regular followees. New posts flow through Kafka so fan-out is fully asynchronous and the post API's latency never depends on follower count. Cassandra is the durable, write-scalable source of truth; Redis is the fast, rebuildable cache that makes reads sub-200ms; and the whole system leans on one property throughout — **a feed is allowed to be a few seconds stale, so almost every hard problem (backlogs, cache misses, cross-region lag) degrades gracefully instead of becoming an outage.**